

Gaiada AI Platform

An AI-native, zero-trust, multi-business operating platform — one interface to run every company, AI woven through every workflow, data processed locally with cloud failover, defended by a continuously hardening security layer. System blueprint, architecture reference, and program status.

DOCUMENT System Blueprint v1.0	SCOPE WS0–WS11 · Compliance · WebDesk	DATE 2026-07-23	OWNER Platform Program
STATE PROTOTYPED · IN PROGRESS	CLASSIFICATION Internal		

Contents

00	Executive Summary
01	Vision & Operating Model
02	System Overview
03	Workstream Map
04	Service Catalog
05	Data, Tenancy & Sync
06	Identity & Zero-Trust
07	AI Layer
08	Automation & Surfaces
09	Security & Resilience
10	Observability & Delivery
11	Deployment & Infra
12	WebDesk — Website Platform
13	Roadmap & Status
14	Decisions & Open Items
15	Glossary

Sections are numbered; each begins on a new page. Use the PDF bookmarks or the page footer to navigate.

§00 Executive Summary

What the platform is, where it stands today, and how the new WebDesk website platform fits in.

The Gaiada AI Platform consolidates every company in the group — digital agency, resort, marine, printing, and more — onto **one modular, multi-tenant system** with a single interface to track all work, and with AI woven through every layer to augment every employee. It is delivered as a pragmatic **Solo-Viable v1** (managed-first, cloud-AI-first, single-region) beneath a preserved, hiring-gated **all-local Target-State** that remains the north star.

Architecturally it is a **layered platform, not a single app**: a NestJS platform core is the source of truth; an MCP Hub exposes its data and tools to AI and surfaces; a Go AI Gateway is the single secured egress for all model calls; an agent brigade, automation engine, and a family of surfaces (WhatsApp/Telegram bot, ERP web UI, department consoles) sit on top. Each component is a **separate standalone deployable**, not a monorepo package — so each is built, run, and scaled independently.

Most of the spine is **prototyped** — running in the dev stack — and in several cases **dev-verified end-to-end** on a live local stack, but **none of it is in production**: the platform core with RBAC and the agency vertical, the event backbone, the Go gateway, the MCP hub, the cross-site sync engine, the observability stack, the automation glue, and a functionally complete ERP UI (in dev) including per-department consoles. What remains is the admin/systems API layer, further verticals, deeper AI-agent capabilities, production deployment, and the target-state infrastructure.

HOW TO READ STATUS IN THIS DOCUMENT

Nothing here is production-finished. Statuses use one vocabulary: **PLANNED** (design only) · **IN PROGRESS** (partial) · **PROTOTYPED** (works in dev, not production) · **DEV-VERIFIED** (prototyped + exercised end-to-end on the local stack). "Built/done/complete" is deliberately avoided. Per-module **versions, status, and future plans** live in the authoritative [docs/modules/MODULES.md](#) registry and [docs/modules/CHANGELOG.md](#).

NEW IN THIS BLUEPRINT – WEBDESK

The Web Dev department is adding **WebDesk**, a centralized multi-tenant website backend (CMS + forms + mail) that serves all client websites through one uniform contract. It is a *new trust zone* attached to this platform: internet-facing, physically separated, and controlled by the ERP. It is **PLANNED** (blueprint only, [v0.0.0](#)). §12 details how it connects; its full engineering spec is a companion blueprint.

§01 Vision & Operating Model

Vision

Migrate Gaiada to AI-assisted operation: enhance every employee's output and consolidate all projects and tasks into **one digital platform with a single interface**. AI augments people; the custom platform is the source of truth; the MCP hub exposes company data and tools to AI and every surface.

Guiding principles

P1 · LOCAL-FIRST, CLOUD-FAILOVER

Production processing on Gaiada's own servers; paid cloud AI only as automatic, capped, alerted failover.

P2 · ONE SOURCE OF TRUTH

Each capability owned by one layer — no logic duplicated across services.

P3 · SHIP SMALL FIRST

Usable tools shipped early for value + discovery while the platform is built in parallel.

P4 · SWAPPABLE EVERYTHING

LLM providers, WhatsApp gateway, data adapters, storage — all behind interfaces.

P5 · ZERO-TRUST BY DEFAULT

mTLS mesh, no public listeners, RBAC + RLS, OBO everywhere, immutable audit.

P6 · SEPARATE DEPLOYABLES

Components are standalone projects, independently built/run/scaled — not a monorepo.

The layer contract (prevents duplicated logic)

- **MCP = access.** Exposes data/tools as nouns & verbs; one authoritative interface per capability.
- **Automation (n8n) = orchestration.** Sequences/schedules by calling MCP tools; no business logic lives in workflows.
- **Custom services = logic.** The platform core owns business rules and the data model.

- **Gateway = AI egress.** Every model call exits through one secured, DLP-enforced chokepoint.

Solo-Viable v1 vs. Target-State

CONCERN	SOLO-VIABLE V1 (NOW)	TARGET-STATE (HIRING-GATED)
Infra	Managed Postgres + container host / small K8s	Per-company physical+VPS pairs, all-local
Multi-site / sync	Central primary + replicas sync engine prototyped	Offline-first sync, regional primaries
AI	Cloud-AI-first via lean Gateway (local optional)	Local frontier models + fine-tuning moat
Identity/authz	Keycloak + Cerbos	Self-hosted IdP, deeper posture
Security	Deterministic zero-trust floor + SIEM	Self-hosted SIEM + AI-SOC (v2)
Surfaces	WhatsApp/Telegram + ERP UI + dept consoles	+ mobile, voice, more
Verticals	Agency (first-deploy), backbone → holding-OS	Resort/marine/print, full multi-tenant

§02 System Overview

The whole estate on one plate — surfaces, the AI layer, the platform core, data, cross-cutting planes, and the adjacent WebDesk zone.

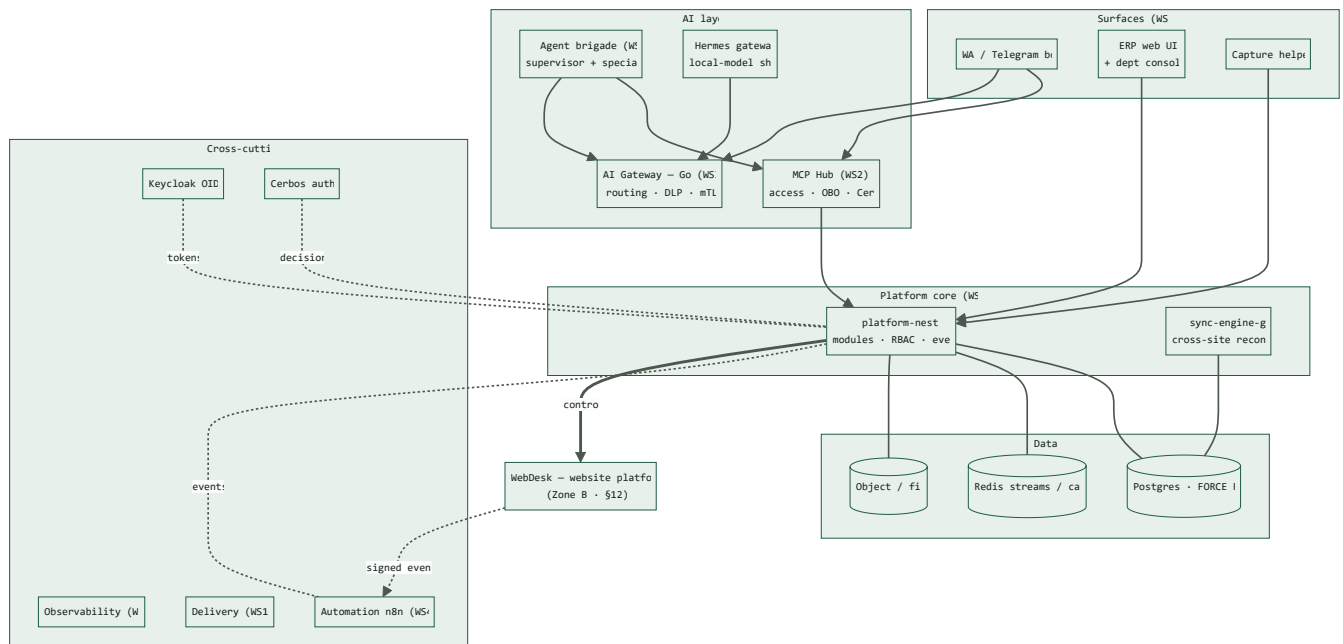


FIG. 02-1 – Master system map. Bold seam = control to the WebDesk zone. Dotted seams = events/tokens/decisions.

- **Surfaces** never assert identity and never hold provider keys — they call the hub/gateway with OBO tokens.
- **The AI layer** is split by responsibility: MCP = access, Gateway = egress, agents = reasoning, Hermes = a local-model shim into the gateway contract.
- **The platform core** is the single source of truth; the sync engine reconciles it across sites.
- **Cross-cutting planes** (identity, authz, observability, delivery, automation) serve every component.
- **WebDesk** is an adjacent, internet-facing zone controlled by the core — detailed in §12.

§03 Workstream Map

The program decomposed into workstreams, each brainstormed → spec'd → built independently, with current status.

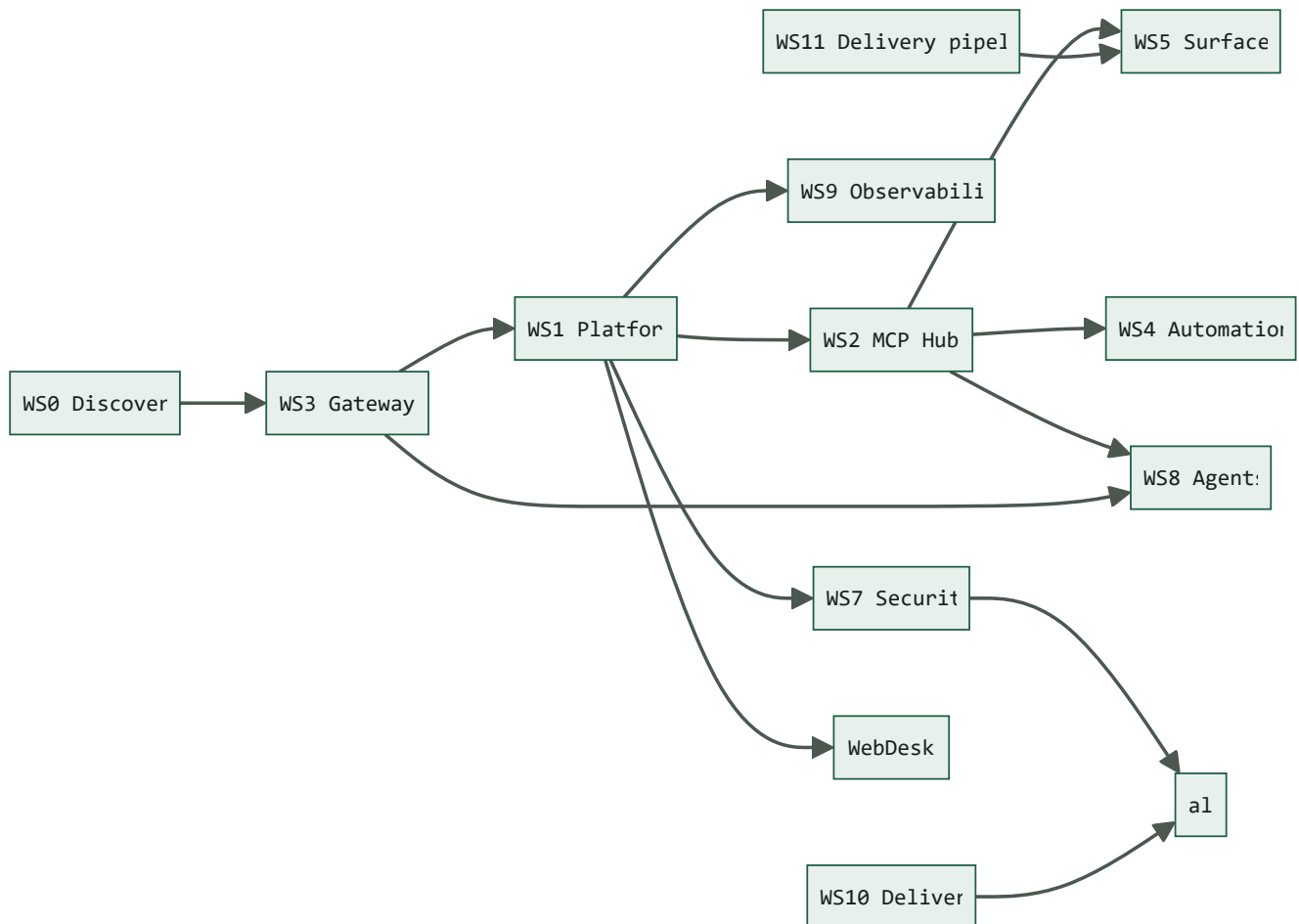


FIG. 03-1 – Workstream dependency graph. WS7 (security) and WS10 (delivery) are cross-cutting.

#	WORKSTREAM	WHAT IT IS	STATUS
0	Discovery	Current-state AI/data/workflow audit (continuous)	ongoing
1	Platform core + sync	Modular multi-tenant NestJS core + Go sync engine	prototyped
2	MCP Hub	Access layer exposing data/tools to AI + surfaces	prototyped
3	AI Gateway (Go)	Provider routing + egress security + DLP	prototyped
4	Automation (n8n)	Workflow orchestration by calling MCP tools	dev-verified
5	Surfaces	WA/TG bot, ERP UI, department consoles	prototyped
6	Governance	Identity, audit, data classification (woven)	woven-in
7	Security & Resilience	Zero-trust floor + HA/DR; AI-SOC is v2	floor prototyped · AI v2
8	AI-Native / Agents	Agent brigade + supervisor + RAG/memory + trainer	in progress (1-3)
9	Observability	OTel metrics/traces/logs, SLOs, alerting	dev-verified
10	Platform Engineering	CI/CD + SBOM/cosign/SLSA; K8s/GitOps target	prototyped (v1)
11	Delivery pipeline	Meeting→MOM→PRD pipeline + capture edge	in progress
C	Compliance	GDPR-grade gate, DPIA/LIA, retention/erasure	drafts
WD	WebDesk	Centralized client-website backend (new)	blueprint

`STATUS = DEV, NOT PRODUCTION`

These statuses describe the *dev* stack. **Per-module versions, detailed status, and future plans are tracked in** `docs/modules/MODULES.md` (registry) and `docs/modules/CHANGELOG.md` — the authoritative, git-tracked source that this blueprint summarizes.

§04 Service Catalog

Each standalone component specified to one shape — purpose, technology, version + status, interfaces, and notes.

REMINDER

Every badge below is a **version + dev status**, not a release. **PROTOTYPED** = works in the dev stack, not production. Authoritative tracking: docs/modules/MODULES.md + CHANGELOG.md .

S-01 platform-nest — Platform Core

V0.5 · PROTOTYPED

PURPOSE	The source of truth: modular multi-tenant data model, RBAC, business logic, event backbone.
TECHNOLOGY	NestJS (Fastify adapter), Postgres with FORCE RLS, Cerbos, Redis Streams. Service on :3004 .
CAPABILITIES	- Core schema + ModuleContract framework (per-tenant enable gate, D17 custom fields). - Cerbos RBAC (scope cascade, decision audit, D11 revocation, PlanResources), D4 identity_links + OBO + dual-proof enrollment, D12 cross-company rollups. - Agency vertical: clients / deliverables / time_entries, campaigns / briefs / creative-asset review, comments, notifications, files (day-one scrubbed). - Event backbone: transactional outbox → Redis Streams relay → consumer w/ dead-letter; HLC-stamped for sync.
OPEN	Admin/systems API layer (/api/admin/* + identity writes) — the top frontend-blocking gap; further verticals + the backbone/ORG-CORE holding-OS build-out.

PURPOSE

The single web interface: one place to run every company and department.

TECHNOLOGY

Next.js, BFF to platform-nest (no direct DB), plain-CSS luxury design system, RBAC-gated nav + company switcher.

CAPABILITIES

- Shell + My Work, cross-company Approvals inbox, Companies/Projects/Tasks, Agency, exec Rollups.
- Systems & Intelligence consoles, Admin (users/identity/modules/compliance/audit), People 360, drag-drop org-structure builder.
- Repsona-style PM + AI tracker, IT device registry + embedded n8n canvas, per-department consoles (Web Dev is the reference template).
- OIDC PKCE login (coexists with dev-login); Playwright e2e; runs backend-free in `DEMO_MODE`.

OPEN

Several sections light up automatically once the backend admin API + org-structure endpoints land.

PURPOSE

The single secured egress for every model call — routing, failover, DLP, cost governance.

TECHNOLOGY

Go, deployed as the `ai-gateway` service on `:3002`.
Byte-for-byte HTTP parity with the retired Node gateway.

CAPABILITIES

- Provider chain w/ failover + circuit breaker (ollama/gemini/claude/echo); `/complete`, `/media`, `/embed`, SSE `/complete/stream`.
- Fail-closed auth + DLP, daily cost cap, egress audit + allowlist (DialContext-enforced), self-signed internal CA + mTLS, site/central topology, DR-burst budget.

OPEN

OpenBao-issued provider creds, media DLP classification, native per-provider token streaming, automated cert rotation.

S-04 mcp-hub — Access Layer

V0.8 · PROTOTYPED

PURPOSE	Expose company data/tools to AI and surfaces via MCP, with OBO auth and deny-by-default policy.
TECHNOLOGY	Official MCP SDK (Streamable HTTP, stateless), fronts platform-nest; Cerbos-authoritative policy.
CAPABILITIES	• Full primitive surface: Tools (core + platform read/write + AI-backed summarize/ocr/vision/transcribe + module-contributed), Resources (<code>gaiada:// ...</code>), Prompts. • OBO principal minting (clients can't assert roles), rate limiting, D11 revocation, mTLS floor, site/central topology, JSONL audit on every decision.
OPEN	OpenBao-minted short-lived creds, Redis-backed multi-instance rate limiting.

S-05 sync-engine-go — Cross-Site Sync

V0.7 · PROTOTYPED

PURPOSE	Reconcile the shared event log across sites with correctness guarantees.
TECHNOLOGY	One Go binary running central (serves push/pull over mTLS) or site (push→pull→GC ticker).
CAPABILITIES	• HLC ordering, declarative per-field conflict resolution (money/status → conflict-queue, else LWW), per-tenant RLS on every op. • Central-authoritative subscription ACL, new-node bootstrap (snapshot + watermark + merkle gate), watermark-gated tombstone GC (delete-wins). • Property-based convergence + partition/chaos tested. Runs idle as <code>sync-central</code> until a real second site exists.

S-06 ai-agents — Agent Brigade

V0.3 · IN PROGRESS

PURPOSE

The "one brain": specialized single-responsibility agents + an orchestrator, working through hub tools + the gateway.

TECHNOLOGY

TS; specialists (status-reporter, approvals-chaser) + supervisor (blackboard, cycle guard, per-goal budget, fan-out cap, approval suspension).

STATUS

Steps 1–3 prototyped incl. pgvector RAG; D14 safety enforced in code. The eval harness is the root gate for further autonomy.

OPEN

Deeper memory/RAG ownership, local-model registry, eval-gated trainer (steps 4–6).

S-07 wa-chat-bot — Messaging Surface

V0.6 · PROTOTYPED

PURPOSE

Work-summary + Q&A/assistant bot over WhatsApp (WAHA) and Telegram.

TECHNOLOGY

Pipeline: webhook (fail-closed) → group registry → PAN/KTP scrub → crypto-shred store → skills/Q&A. 12:00/18:00 digests, media enrichment via gateway.

STATUS

Prototyped (trial-lite): Phases 0–3 + P5a features; Telegram live in dev, WAHA primary once its number is scanned. Blocked on infra + legal Gate 1 before real ingestion. Never holds keys, never asserts identity.

S-08 hermes-gateway & capture-helper

V0.2 · PROTO / IN PROGRESS

HERMES-GATEWAY

A shim that makes a local Hermes model the bot's brain by speaking the Gateway contract — no bot changes; verified headless.

CAPTURE-HELPER

WS11 capture edge: in-ERP record → local Whisper
`.txt` → ingest → Shared Drive; feeds the meeting→MOM→PRD delivery pipeline.

S-09 automation, observability & infra

V0.4-0.6 · DEV-VERIFIED / PROTOTYPED

AUTOMATION (WS4)

n8n + MCP-calling templates; scoped n8n accounts, impact gate, event bridge, approvals-suspension surface. Dev-verified — 3 flows exercised e2e on the local stack.

OBSERVABILITY (WS9)

OTel across all services (fail-soft), opt-in Grafana/Prometheus/Tempo/Loki stack, multi-burn-rate SLOs, alerting, restore drills. Verified end-to-end.

INFRA (WS10)

Full VPS Docker Compose stack, CI, crypto-shred-safe backups, supply-chain pipeline (SBOM + cosign + SLSA). K8s/GitOps/SPIFFE are target-state.

§05 Data, Tenancy & Sync

One shared database, hard per-tenant isolation, module-driven heterogeneity, and an event log that doubles as the sync spine.

Multi-tenancy & modules

- **Shared DB + FORCE RLS** on an authorized-tenant-set (D5) — every query is tenant-scoped; the app role is `NOBYPASSRLS`.
- **Heterogeneity via compile-time modules + custom fields** (D17) — verticals extend the core through the `ModuleContract` framework and a per-tenant enable gate, not schema forks.
- **Per-service DB roles** — owner/migrator + per-service runtime roles; a hard platform↔sync grant boundary; Keycloak/n8n on Postgres; isolated bot Postgres; split Redis.

Event backbone & sync

A transactional **outbox** is relayed to Redis Streams and consumed with dead-lettering; the same `outbox_events` log is the sync spine. platform-nest stamps a **Hybrid Logical Clock** on every emit, and the Go sync engine reconciles across sites with per-field conflict rules and a dedup ledger that never touches the relay's own state (D7).

HA ≠ BACKUP

Streaming replication provides availability; durability comes from immutable/WORM backups + PITR and rehearsed restores (§10). Crypto-shred (two-axis subject×entity) makes PII erasure real.

§06 Identity, RBAC & Zero-Trust

Authentication and authorization are split and both authoritative: **Keycloak** (OIDC) proves who you are; **Cerbos** decides what you may do; **RLS** enforces which rows you touch. No surface asserts identity — everything acts **on-behalf-of** a minted principal.

AUTHN

Keycloak OIDC (JWKS verify, auto-provision, email-verified takeover guard). Hybrid: SSO + dev-login coexist; UI uses PKCE.

AUTHZ

Cerbos versioned policy repo — scope-cascade derived roles, PlanResources, team scope, decision audit, D11 revocation.

ISOLATION

Postgres FORCE RLS on every table; per-service runtime roles; OBO principal minting at the hub.

MESH

Self-signed internal CA + mTLS between gateway/hub/sync; peer-CN allowlists; no public listeners on datastores.

ENROLLMENT

D4 dual-proof identity_links (WA/TG ↔ platform user), step-up ceilings for sensitive skills.

UI RBAC

`lib/rbac.ts` capability model + company switcher; nav + write actions gated (Cerbos/RLS remain authoritative).

§07 AI Layer

Four responsibilities, four owners — so no layer duplicates another.

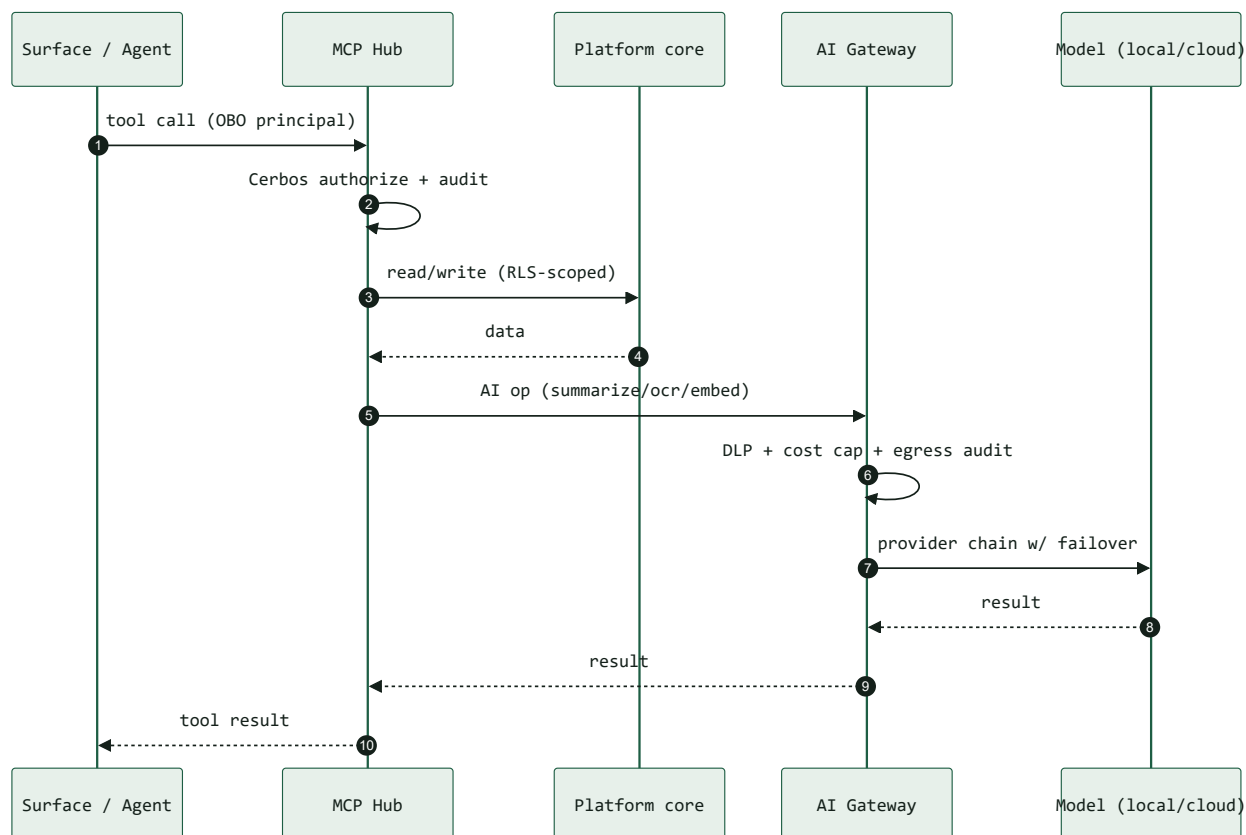


FIG. 07-1 — A typical AI request: access (hub) → logic (core) → egress (gateway) → model, each enforcing its own controls.

- **Gateway (WS3)** — the only path to any model; DLP, cost cap, failover, egress audit.
- **MCP Hub (WS2)** — the only path to company data/tools; OBO + Cerbos + audit.
- **Agents (WS8)** — reasoning/orchestration over hub tools, budget- and safety-bounded.
- **Local inference** — Ollama/llama.cpp + Hermes via the gateway contract; cloud is capped failover. The whole stack can run offline.

§08 Automation & Surfaces

Automation (WS4)

n8n orchestrates by **calling MCP tools** — no business logic in workflows. Scoped n8n accounts, an impact gate, a platform→n8n event bridge, and an approvals-suspension surface are prototyped; three flows are dev-verified end-to-end on the local stack. Temporal is reserved for when a genuinely durable flow exists.

Surfaces (WS5)

- **Messaging** — WhatsApp (WAHA) + Telegram work-summary/assistant bot.
- **ERP web UI** — the primary interface; functionally complete in dev across Plans 1–5 + People + org builder (backend-dependent sections degrade gracefully).
- **Department consoles** — a reusable department-first template (Web Dev prototyped as the reference), with work-detection and integrations on the Web Dev roadmap.
- **Delivery pipeline (WS11)** — meeting capture → MOM → PRD/report/scope, feeding the surfaces.

HOLDING-OS DIRECTION (BACKBONE PROGRAM)

The approved backbone program upgrades org/module/RBAC toward a true holding-OS: shared-service departments serving N companies via a drag-drop org chart, per-company data slicing, and an HR company-selector — a modular monolith, payroll excluded.

§09 Security & Resilience

Security is a workstream (WS7), not an afterthought: a **deterministic zero-trust floor** now, an **AI security layer** as v2.

LAYER	CONTROL	STATE
Network	mTLS mesh, peer allowlists, no public listeners on datastores, egress allowlist	prototyped
Identity/authz	Keycloak + Cerbos + FORCE RLS + OBO + revocation	prototyped
Data protection	Crypto-shred (two-axis), PAN/KTP ingestion scrubber, encryption at rest	prototyped
Egress/AI	Gateway DLP, cost cap, egress audit, fail-closed auth	prototyped
Resilience	Streaming replication + WORM backups + PITR + restore drills; DR-burst budget	prototyped
SIEM / AI-SOC	Anomaly detection, adaptive response, rogue-employee isolation	v2
Compliance	DPIA/LIA/notices, retention/erasure/DSR, classification	drafts (legal)

GATES BEFORE REAL DATA

Do not ingest real employee data until both the legal Gate 1 (lawyer-reviewed pack) and the technical day-one gate (crypto-shred + scrubber) are green.

§10 Observability & Delivery

Observability (WS9)

All services carry **OpenTelemetry** (traces + metrics + trace-correlated logs, W3C propagation, fail-soft — no-op unless `OTEL_ENABLED`). An opt-in self-hosted stack — OTel Collector → Prometheus/Tempo/Loki + Grafana + Alertmanager + exporters — provides exec + per-workstream dashboards, multi-burn-rate SLOs, blackbox synthetics, ≥ 2 independent alert transports, and a measured restore drill. Dev-verified end-to-end on a live local stack (not yet deployed to production).

Delivery (WS10)

A supply-chain-secure release pipeline (**SBOM + cosign + SLSA**), per-component Dockerfiles, a full VPS Compose stack, local CI (`test-all.sh`), and crypto-shred-safe nightly backups. Kubernetes/GitOps and SPIFFE/SPIRE workload identity are the hiring-gated target-state.

§11 Deployment & Infra

One Compose estate today; the whole stack has run locally with 16 healthy containers.

SERVICE	ROLE	PORT / NOTE
platform-nest	Platform core / BFF	:3004
platform-ui	ERP web UI	Next.js
ai-gateway	AI egress (Go)	:3002
mcp-hub	Access layer	Streamable HTTP
cerbos	Authz	:3592 / :3593
keycloak	OIDC IdP	issuer/JWKS split
postgres xN	Platform + isolated bot instance	FORCE RLS
redis (split)	Streams + cache	split by concern
waha	WhatsApp gateway	free since 2026.6.1
whisper / knowledge	Transcription / RAG	local
bot + media-worker	Messaging surface	BullMQ
sync-central	Cross-site sync	idle until 2nd site
n8n	Automation	on Postgres

NOTES

Cerbos must run with published ports or all authz fails. hermes-gateway is intentionally non-dockerized (local model shim). On Windows, Smart App Control blocks locally-compiled Go exes — Go services build/run via WSL. The stack runs fully offline with local Ollama.

§12 WebDesk — Website Platform

How the new centralized client-website backend relates to this platform.

WebDesk is the Web Dev department's centralized, multi-tenant website backend — one CMS + forms + mail platform that all client sites (WordPress, Astro, Node) consume through a uniform, versioned contract, so frontend engineers build only frontends. It is *not* part of the ERP core; it is a **separate, internet-facing trust zone (Zone B)** attached to the platform (Zone A).

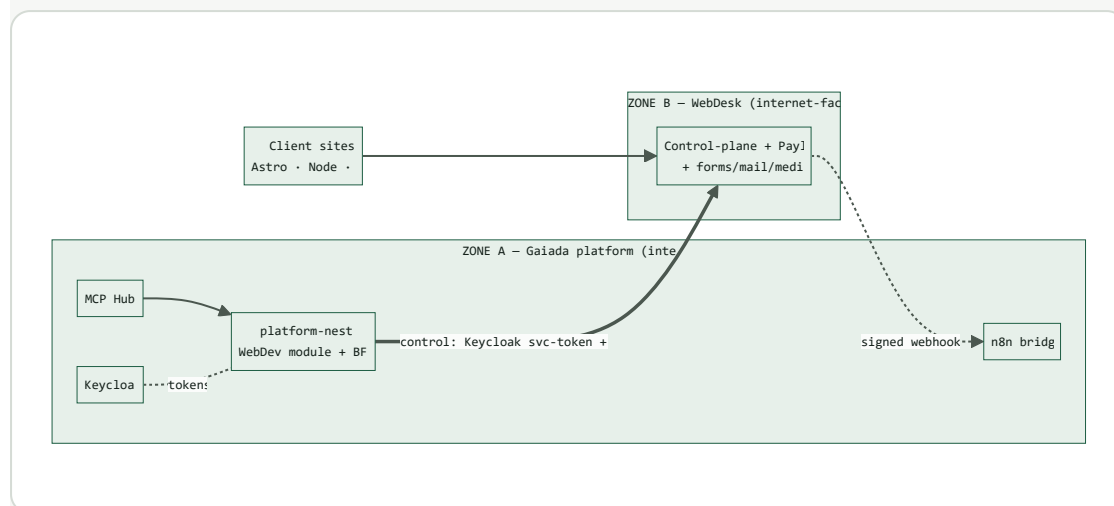


FIG. 12-1 – WebDesk attaches as Zone B. The ERP controls it one-way; client sites never reach Zone A.

What WebDesk reuses from this platform

PLATFORM CAPABILITY	WEBDESK REUSE
Keycloak (WS1/6)	Client-credentials + mTLS on the ERP→WebDesk control channel
MCP Hub (WS2)	Control-plane commands exposed as MCP tools for AI operation
WS4 approvals	Gate irreversible WebDesk actions (promote-to-live, delete, key rotation)
Cerbos + RLS pattern	Per-tenant isolation inside WebDesk
n8n event bridge (WS4)	Ingest WebDesk events (form received, deploy done) back into the ERP
WS9 observability	OTel from WebDesk exported to the same Grafana stack
WS10 delivery	SBOM/cosign pipeline + promotion/rollback releases
ERP WebDev console (WS5)	The operator UI: site registry, provisioning, promote/rollback

RELATIONSHIP IN ONE LINE

The Gaiada platform is the **internal brain and control plane**; WebDesk is an **internet-facing execution zone** it commands. Full details: the companion *GAIADA WebDesk — Engineering Blueprint*.

§13 Roadmap & Current Status

Where we are

The architectural spine (WS1/2/3/7/9/10) is **prototyped** and, for the automation + observability paths, **dev-verified** end-to-end on a live local stack — but not in production. The ERP UI is functionally complete in dev and wired to a running dev backend. The near-term critical path is the **backend admin/systems API** (unblocks the placeholder UI sections), the **backbone/ORG-CORE holding-OS** build-out, deeper **AI-agent** capabilities, a first **production deploy**, and standing up **WebDesk**.

A Spine — prototyped

Platform core + RBAC + events, Go gateway, MCP hub, sync engine, observability, delivery v1, ERP UI.

STATUS → prototyped in dev · parts dev-verified on a live local stack
· NOT in production.

B Backend admin/systems API

`/api/admin/*` + identity writes + org-structure endpoints — the top frontend-blocking gap.

GATE → the ERP's Systems/Intelligence/Admin sections light up.

C Backbone → holding-OS (ORG-CORE)

Shared-service depts serving N companies, drag-drop org chart, per-company slicing.

GATE → the group operates as one holding-OS.

D AI-agent depth (WS8 steps 4–6)

Memory/RAG ownership, local-model registry, eval-gated trainer.

GATE → agents pass the eval harness for expanded autonomy.

E WebDesk platform

Stand up the website backend per its companion blueprint (P1→P6).

GATE → client sites served centrally; web3forms retired.

F Target-state (hiring-gated)

All-local infra, self-hosted IdP/SIEM + AI-SOC, local frontier models, more verticals.

Opens as headcount grows.

§14 Decisions & Open Items

Locked decisions

#	DECISION
L1	Solo-Viable v1 now; all-local Target-State preserved, hiring-gated
L2	Layer contract: MCP=access, n8n=orchestration, services=logic, Gateway=AI egress
L3	Multi-tenant shared DB + FORCE RLS; heterogeneity via modules + custom fields
L4	Keycloak + Cerbos + RLS + OBO everywhere
L5	Two-tier replication; HA ≠ backup → WORM backups + PITR
L6	Go is THE AI gateway (Node retired 2026-07-14)
L7	Agency is a first-deploy child company; agent trainer is eval-gated + human-approved
L8	Components stay separate deployables, not a monorepo
L9	Full-fidelity mandate — no new shortcuts without an explicit decision
L10	WebDesk is a separate internet-facing trust zone controlled by the ERP

Open items

ITEM	OWNER / GATE
Backend admin/systems API + identity writes	platform-nest
Backbone/ORG-CORE holding-OS build-out	platform
AI-agent steps 4–6 (eval harness is the gate)	ai-agents
Compliance Gate 1 (lawyer review) before real data	legal
WebDesk stand-up + its five open design items	WebDesk blueprint
Target-state infra (K8s/GitOps/SPIFFE, self-hosted IdP/SIEM)	hiring-gated

§15

Glossary

TERM	MEANING
WS0-WS11	Workstreams — the program's independently-developed units (§03)
PROTOTYPED	Works in the dev stack; not production-complete (see §00 status vocabulary)
DEV-VERIFIED	Prototyped and exercised end-to-end on the local stack; still not production
OBO	On-behalf-of — a surface acts as a minted principal, never asserting its own identity
RLS	Postgres Row-Level Security — per-tenant query scoping (FORCE, NOBYPASSRLS role)
HLC	Hybrid Logical Clock — stamps event order for cross-site sync
MCP	Model Context Protocol — the hub's access interface for AI/surfaces
DLP	Data-loss prevention — enforced at the AI gateway egress
Crypto-shred	Per-subject×entity key deletion making PII erasure real
Zone A / Zone B	Internal platform / internet-facing WebDesk trust zones (§12)
Target-State	The preserved all-local north star, unlocked by hiring

Gaiada AI Platform — System Blueprint v1.0

Internal · 2026-07-23 · Platform Program. Status reflects the code (README/CLAUDE.md) + program memory as of this date; the running code is the source of truth. Companion: GAIADA WebDesk – Engineering Blueprint.